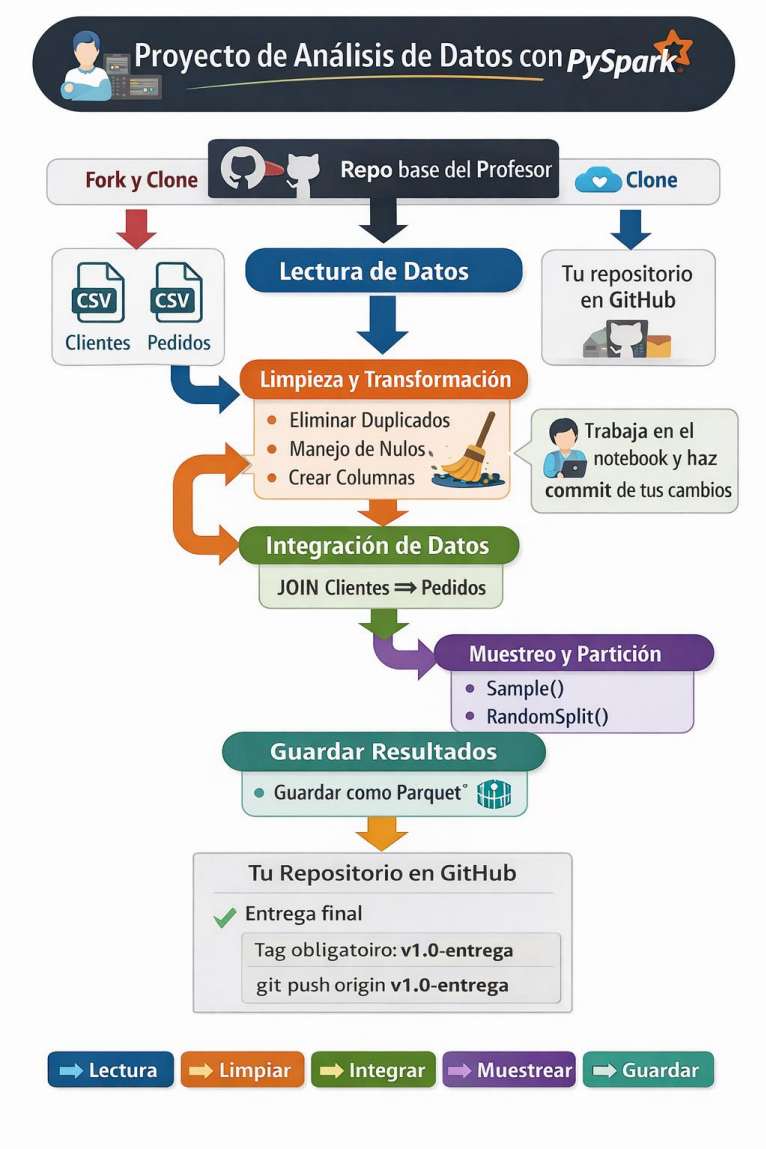
Proyecto individual — Análisis de datos con Spark DataFrames

Trabajo sobre repositorio GitHub y entorno dockerizado de clase



CEIABD: Sistemas de Big Data

14/04/2026

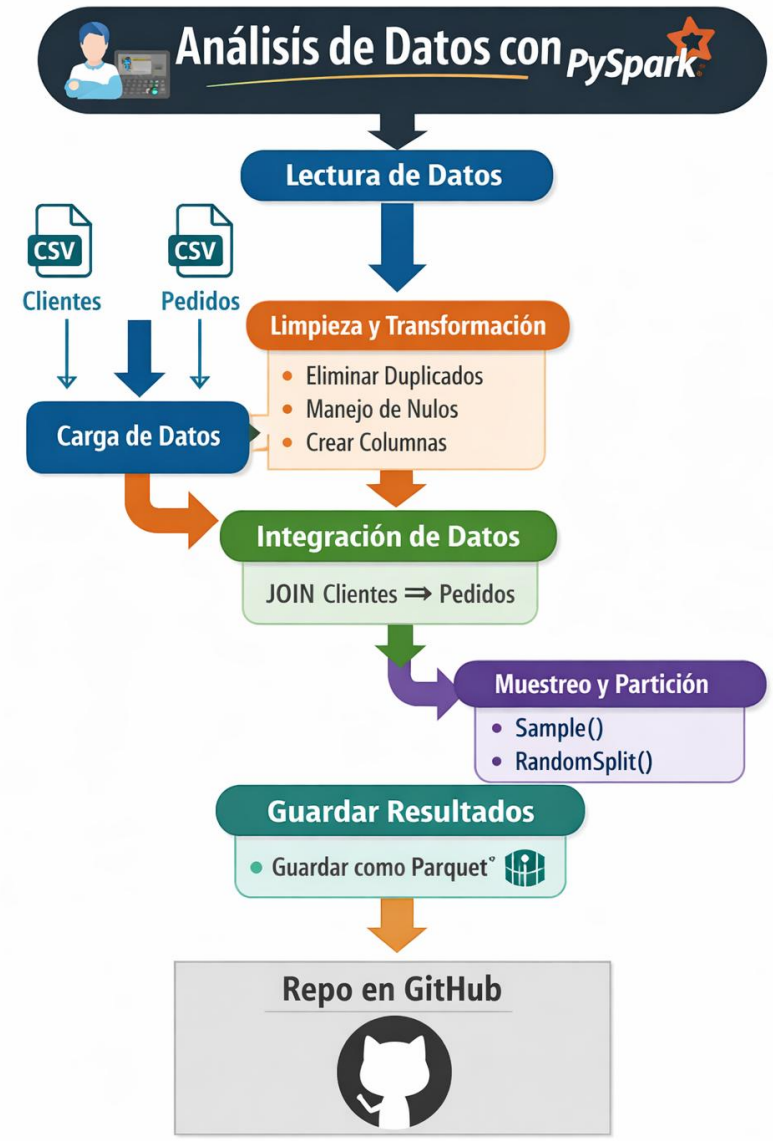
ÍNDICE

Descripción 3
Contexto 4
Repositorio y entorno de trabajo 4
Cómo levantar el entorno 4
Servicios disponibles 4
Flujo de trabajo obligatorio con GitHub 5
Notebook que debes completar 5
Tareas a realizar 5
1. Lectura de datos 5
2. Limpieza de datos 5
3. Transformación 5
4. Integración de datos 5
5. Análisis 6
6. SQL 6
7. Muestreo y partición 6
8. Persistencia 6
Entregables 6
Criterios de evaluación 6
Evidencias recomendadas 7
Notas 7

Descripción

En esta práctica realizarás un trabajo individual utilizando Apache Spark y la DataFrame API en PySpark. Trabajarás sobre un repositorio de GitHub mediante fork y ejecutarás la práctica en el entorno dockerizado incluido en el propio repositorio.

El objetivo es desarrollar un flujo completo de trabajo que incluya lectura de datos, limpieza, transformación, combinación de fuentes, análisis con agregaciones, uso de SQL y persistencia de resultados.



Contexto

Una pequeña empresa de comercio electrónico dispone de dos fuentes de datos: un fichero de clientes y un fichero de pedidos. Ambos contienen información relevante para el análisis del negocio, pero presentan problemas habituales en entornos reales: duplicados, valores nulos, inconsistencias de formato y registros sin correspondencia entre tablas.

Tu tarea será preparar los datos y obtener información útil para la toma de decisiones.

Repositorio y entorno de trabajo

El repositorio (https://github.com/josedavidmi/spark-dataframes-lab.git) incluye una carpeta llamada spark_jupyter con todo lo necesario para montar el entorno dockerizado de clase.

- docker-compose-jupyter.yml para levantar Spark Master, dos workers, History Server y JupyterLab.
- Dockerfile.jupyter para construir la imagen del notebook.
- notebooks/iniciar_spark.py para crear la SparkSession desde el cuaderno.
- apps/datos/ para los ficheros de entrada.
- apps/salida/ para los resultados generados.

Cómo levantar el entorno

1. Abre una terminal y sitúate dentro de la carpeta spark_jupyter.

2. Ejecuta el siguiente comando:

```
docker compose -f docker-compose-jupyter.yml up -d --build
```

3. Espera a que los contenedores arranquen correctamente.

Servicios disponibles

Servicio	Acceso
JupyterLab	http://localhost:8888 | token: spark
Spark Master UI	http://localhost:8080
Spark Worker 1 UI	http://localhost:8081
Spark Worker 2 UI	http://localhost:8082
Spark History Server	http://localhost:18080

Flujo de trabajo obligatorio con GitHub

1. Haz Fork del repositorio del profesor. https://github.com/josedavidmi/spark-dataframes-lab.git
2. Clona tu fork en local.
3. Levanta el entorno dockerizado incluido en el repositorio.
4. Completa el notebook de la práctica.
5. Realiza commits progresivos con los cambios.
6. Sube los cambios a tu repositorio.
7. Crea el tag obligatorio de entrega v1.0-entrega.

Notebook que debes completar

Debes resolver el cuaderno:

spark_jupyter/notebooks/practica_clientes_pedidos.ipynb

Los datos están montados dentro del contenedor en las siguientes rutas:

- /opt/spark-apps/datos/clientes.csv
- /opt/spark-apps/datos/pedidos.csv

Tareas a realizar

1. Lectura de datos

- Cargar los ficheros CSV de clientes y pedidos.
- Mostrar esquema y contenido.

2. Limpieza de datos

- Eliminar duplicados en clientes.
- Eliminar espacios en columnas de texto.
- Gestionar valores nulos.
- Validar tipos de datos.

3. Transformación

- Crear nuevas columnas, por ejemplo importe = cantidad × precio_unitario.
- Convertir tipos si es necesario.

4. Integración de datos

- Realizar un join entre clientes y pedidos.
- Analizar qué registros se pierden y por qué.

5. Análisis

- Filtrar datos relevantes.
- Clasificar pedidos con when.
- Calcular métricas: número de pedidos, ingresos totales y medias.

6. SQL

- Crear una vista temporal.
- Realizar al menos una consulta SQL significativa.

7. Muestreo y partición

- Aplicar sample().
- Aplicar randomSplit().

8. Persistencia

- Guardar resultados en formato Parquet.
- Leerlos de nuevo.

Entregables

- Repositorio GitHub (fork) con el trabajo realizado.
- Notebook completamente ejecutado.
- Evidencias incluidas en docs/evidencias.md.
- Tag obligatorio de entrega:

git tag v1.0-entrega
git push origin v1.0-entrega

Criterios de evaluación

Criterio	Descripción	Puntos
Lectura y comprensión de datos	Lectura correcta de CSV, uso de opciones e inspección de esquema y contenido.	1,50
Limpieza de datos	Duplicados, nulos, trim y coherencia de la limpieza aplicada.	1,50
Transformaciones	Uso correcto de withColumn, cálculo de nuevas columnas y cast.	1,50
Integración de datos (JOIN)	Join correcto, elección del tipo de join e interpretación	2,00

	del resultado.	
Análisis y agregaciones	Uso de groupBy, cálculo correcto de métricas y ordenación.	2,00
SQL sobre DataFrames	Creación de vista temporal y consulta SQL correcta.	0,50
Muestreo y partición	Uso correcto de sample() y randomSplit().	0,50
Persistencia de datos	Escritura en Parquet y lectura posterior correcta.	0,50
Calidad del código y presentación	Código claro, ordenado y con explicaciones comprensibles.	0,50

TOTAL: 10,00 puntos

Evidencias recomendadas

- Captura de JupyterLab funcionando.
- Captura del Spark Master UI.
- Esquema y muestra inicial de los DataFrames.
- Resultado de la limpieza.
- Resultado del join y explicación de registros perdidos.
- Agregaciones obtenidas.
- Consulta SQL realizada.
- Escritura y lectura del resultado en Parquet.

Notas

Se valorará la claridad del código, la comprensión del proceso y la correcta ejecución del notebook. Se recomienda trabajar de forma incremental y ejecutar el cuaderno poco a poco para comprobar el resultado de cada bloque.